

Universidad Tecnológica Nacional

Trabajo Práctico Integrador - Programación 2

Proyecto: Food Store

Sistema de Gestión de Pedidos de Comida (Consola + JDBC)

Estudiantes: Xavier Pappalardo, Lautaro Fernández,
Lautaro Gómez, Daniela Diaz

Materia: Programación 2

Grupo : N° 5

Fecha de entrega: 22 de junio de 2026

Índice

1. Introducción	3
2. Marco Teórico	4
3. Descripción del Proyecto	6
4. Decisiones Técnicas y Arquitectura	7
5. Dificultades y Soluciones	11
6. Conclusiones	11
7. Bibliografía y Webgrafía	12
8. Anexos: Repositorio	12

1. Introducción

El presente Trabajo Práctico Integrador corresponde a la asignatura Programación 2 y tiene como finalidad aplicar los conocimientos adquiridos durante el cursado mediante el desarrollo de una aplicación de consola en Java.

El proyecto desarrollado se denomina “Food Store”, un sistema orientado a la gestión de pedidos de comida que permite administrar categorías, productos, usuarios y pedidos. La aplicación fue construida utilizando los principios de la Programación Orientada a Objetos (POO), incorporando herencia, encapsulamiento, polimorfismo, interfaces, colecciones y manejo de excepciones.

El sistema permite realizar operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre las entidades principales mediante un menú interactivo en consola, almacenando toda la información en memoria durante la ejecución del programa.

2. Marco Teórico

2.1 Java

Java es un lenguaje de programación orientado a objetos, multiplataforma y ampliamente utilizado en el desarrollo de aplicaciones empresariales, de escritorio, web y móviles.

Una de sus principales ventajas es la portabilidad, ya que los programas compilados pueden ejecutarse en cualquier sistema operativo que disponga de una Máquina Virtual de Java (JVM).

Para el desarrollo de este proyecto se utilizó Java 21, aprovechando sus características modernas y su robustez para la implementación de aplicaciones orientadas a objetos.

2.2 Programación Orientada a Objetos

La Programación Orientada a Objetos es un paradigma de desarrollo basado en la representación de entidades del mundo real mediante objetos.

Durante el desarrollo del sistema se aplicaron los siguientes conceptos:

Encapsulamiento : Permite proteger los datos internos de una clase mediante atributos privados y métodos públicos de acceso.

Herencia: Facilita la reutilización de código permitiendo que una clase herede atributos y comportamientos de otra.

Polimorfismo: Posibilita que diferentes objetos respondan de manera distinta a una misma operación.

Abstracción: Consiste en representar únicamente las características esenciales de una entidad, ocultando detalles innecesarios.

2.3 Interfaces

Las interfaces permiten definir comportamientos comunes que pueden ser implementados por distintas clases.

En el proyecto se implementó la interfaz Calculable para establecer el método encargado de calcular el total de un pedido.

2.4 Colecciones

Las colecciones de Java permiten almacenar y gestionar grupos de objetos dinámicamente. Para este proyecto se utilizaron estructuras como ArrayList para almacenar:

- Categorías
- Productos
- Usuarios
- Pedidos
- Detalles de pedidos

2.5 Manejo de Excepciones

El manejo de excepciones permite controlar errores durante la ejecución del programa evitando cierres inesperados.

Se utilizaron excepciones para validar:

- Precios negativos.
- Stock insuficiente.
- Correos electrónicos duplicados.
- Identificadores inexistentes.
- Datos ingresados incorrectamente.

3. Descripción del Proyecto

Food Store es una aplicación de consola diseñada para gestionar pedidos de comida. El sistema fue desarrollado siguiendo el modelo UML proporcionado por la cátedra y permite administrar las principales entidades involucradas en el proceso de venta.

Las funcionalidades implementadas incluyen:

Gestión de Categorías

- Listar categorías.
- Crear categorías.
- Modificar categorías.
- Eliminar categorías mediante baja lógica.

Gestión de Productos

- Listar productos.
- Crear productos.
- Modificar productos.
- Eliminar productos mediante baja lógica.

Gestión de Usuarios

- Listar usuarios.
- Crear usuarios.
- Modificar usuarios.
- Eliminar usuarios mediante baja lógica.

Gestión de Pedidos

- Crear pedidos.
- Agregar detalles de pedido.
- Calcular totales automáticamente.
- Modificar estado del pedido.
- Modificar forma de pago.
- Eliminar pedidos mediante baja lógica.

Toda la información se almacena en memoria utilizando colecciones, por lo que los datos permanecen disponibles únicamente durante la ejecución del programa.

4. Decisiones Técnicas y Arquitectura

Para lograr una estructura organizada y fácil de mantener, el proyecto fue dividido en diferentes paquetes.

Estructura General

src/

|— Main.java

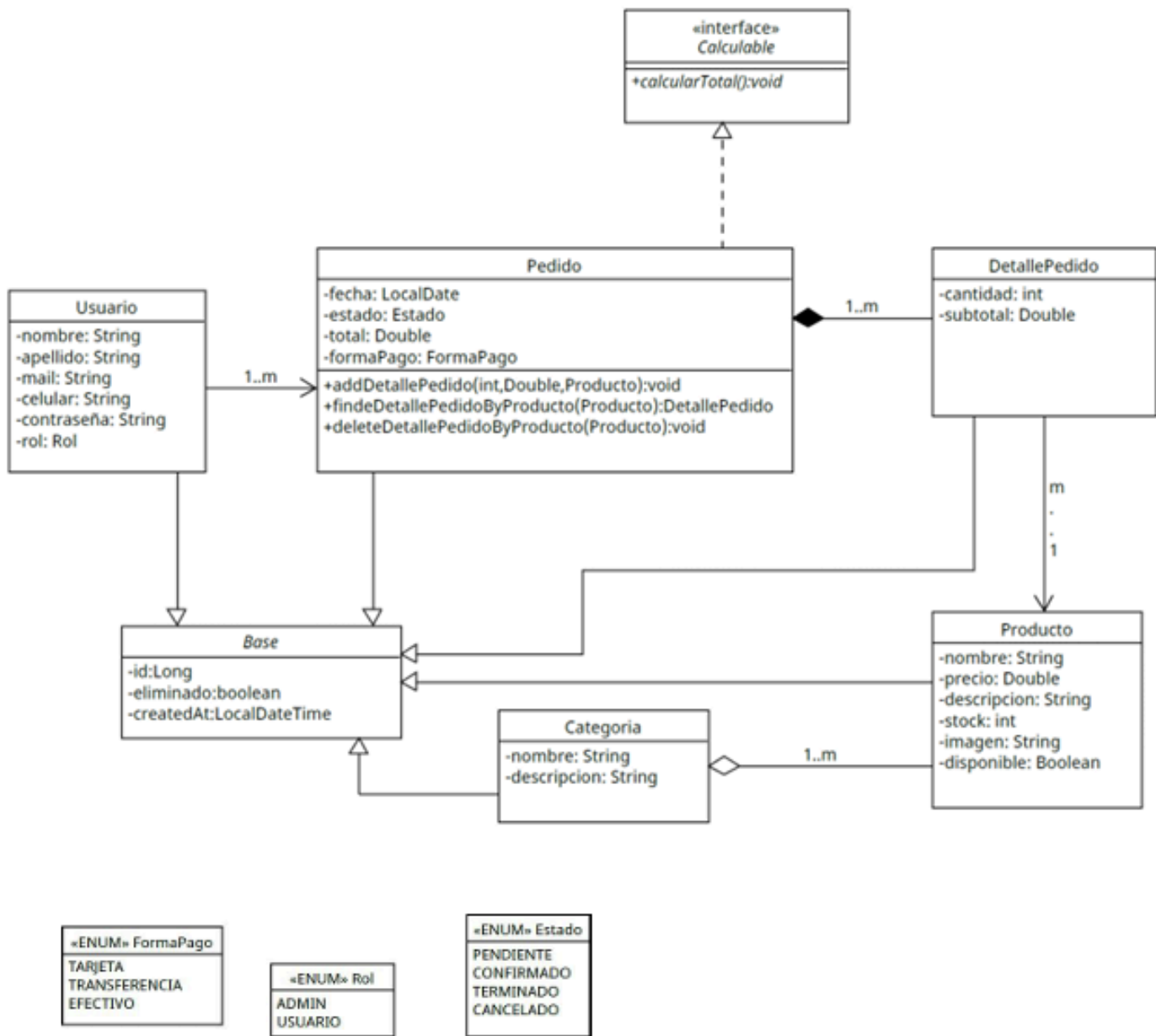
|— entities/

|— enums/

|— exception/

|— interfaces/

Modelo (UML) y Entidades



Entidades

Las entidades representan los objetos principales del sistema.

Base:

Clase abstracta que contiene:

- id
- eliminado
- createdAt

Categoría:

Representa las categorías de productos.

Producto:

Representa los productos comercializados.

Usuario:

Representa a los clientes u operadores del sistema.

Pedido:

Representa una compra realizada por un usuario.

DetallePedido:

Representa cada producto incluido dentro de un pedido.

Menú

MenuCRUDCategorias:

Menú donde se pueden listar, crear, editar y eliminar categorías de productos.

MenuCRUDPedidos:

Menú donde se pueden listar, crear, editar y eliminar pedidos.

MenuCRUDProductos:

Menú donde se pueden listar, crear, editar y eliminar productos.

MenuCRUDUsuarios:

Menú donde se pueden listar, crear, editar y eliminar usuarios.

UsuarioLog:

Menú que permite que los usuarios ingresen con su id o se creen nuevos usuarios.

Services

CategoriaService:

Contiene los métodos para las acciones que puede realizar el MenuCRUDCategorias.

PedidoService:

Contiene los métodos para las acciones que puede realizar el MenuCRUDPedidos.

ProductoService:

Contiene los métodos para las acciones que puede realizar el MenuCRUDProductos.

UsuarioService:

Contiene los métodos para las acciones que puede realizar el MenuCRUDUsuarios.

Enumeraciones

Se utilizaron enumeraciones para restringir valores válidos:

Rol

- ADMIN
- USUARIO

Estado

- PENDIENTE
- CONFIRMADO
- TERMINADO
- CANCELADO

FormaPago

- TARJETA
- TRANSFERENCIA
- EFECTIVO

Interfaces

Se implementó la interfaz Calculable, utilizada por la clase Pedido para calcular automáticamente el importe total de la compra.

5. Dificultades y Soluciones

- **Organización del Código:**

Al aumentar la cantidad de funcionalidades, mantener el código ordenado se volvió fundamental. Por este motivo se separó la aplicación en capas y paquetes específicos, permitiendo una mejor distribución de responsabilidades.

-

5. Conclusiones

El desarrollo del sistema Food Store permitió aplicar de forma práctica los conocimientos adquiridos durante la cursada de Programación 2.

A través de este proyecto se trabajó con conceptos fundamentales como Programación Orientada a Objetos, herencia, polimorfismo, interfaces, colecciones, enumeraciones y manejo de excepciones.

Asimismo, se fortalecieron habilidades relacionadas con la organización de proyectos, la implementación de reglas de negocio y la validación de datos.

El resultado final es una aplicación funcional capaz de gestionar categorías, productos, usuarios y pedidos mediante una interfaz de consola simple y organizada.

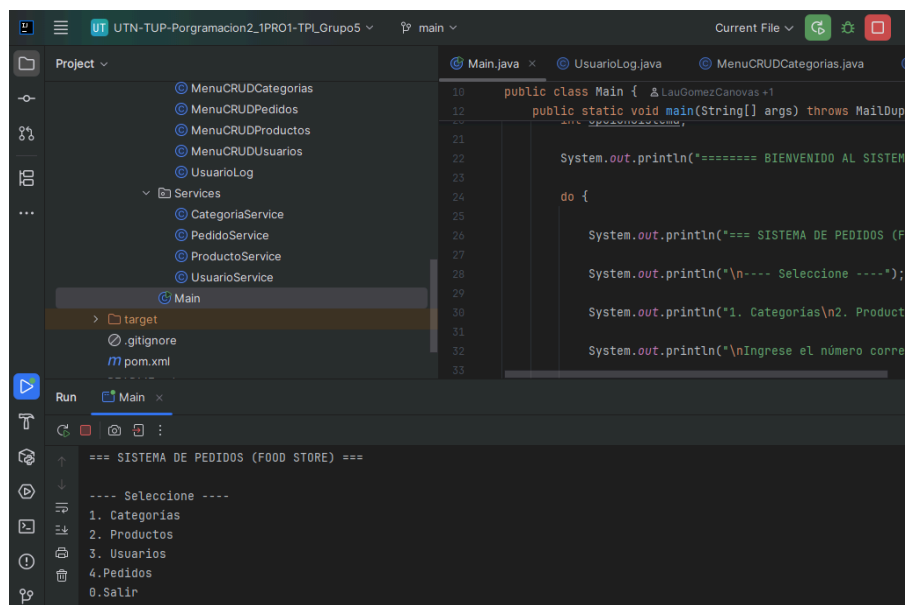
6. Bibliografía / Webgrafía

- Oracle. Java Documentation.
- <https://docs.oracle.com>
- Oracle. Java Collections Framework.
- <https://docs.oracle.com/javase>
- GitHub Documentation.
- <https://docs.github.com>
- Material de estudio de Programación 2.
- Apuntes y presentaciones proporcionadas por la cátedra.

7. ANEXOS

Repositorio del Proyecto (GitHub): [Repositorio GitHub Trabajo Práctico Integrador Programación 2 Grupo 5](#)

Capturas del Sistema:



The screenshot displays an IDE window for a Java project named 'UTN-TUP-Programacion2_1PRO1-TPLGrupo5'. The project structure on the left includes packages like 'MenuCRUDCategorias', 'MenuCRUDPedidos', 'MenuCRUDProductos', 'MenuCRUDUsuarios', 'UsuarioLog', and 'Services'. The 'Main' class is selected. The right pane shows the code for 'Main.java', which includes a 'main' method that prints a welcome message and a menu. The bottom pane shows the output of the program, which displays the same welcome message and a menu with options: 1. Categorias, 2. Productos, 3. Usuarios, 4. Pedidos, and 0. Salir.

```
public class Main {
    public static void main(String[] args) throws MailDupl
    {
        System.out.println("===== BIENVENIDO AL SISTEMA");
        do {
            System.out.println("=== SISTEMA DE PEDIDOS (FOOD STORE) ===");
            System.out.println("---- Seleccione ----");
            System.out.println("1. Categorias\n2. Productos\n3. Usuarios\n4. Pedidos\n0. Salir");
        } while (true);
    }
}
```

```
===== BIENVENIDO AL SISTEMA
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
---- Seleccione ----
1. Categorias
2. Productos
3. Usuarios
4. Pedidos
0. Salir
```

```
Run Main x
4.Pedidos
0.Salir

Ingrese el número correspondiente a la acción que desee ejecutar:
1

No se han encontrado usuarios registrados. ¿Desea crear uno?
(Ingrese '1' si la respuesta es afirmativa, o '0' en caso contrario):
```

```
Run Main x
1

No se han encontrado usuarios registrados. ¿Desea crear uno?
(Ingrese '1' si la respuesta es afirmativa, o '0' en caso contrario):
1

Ingrese el id del nuevo usuario:
1

Ingrese el nombre del nuevo usuario:
Pepe
```

```
Run Main x
pepe@gmail.com

Ingrese el celular del nuevo usuario:
123456789

Ingrese la contraseña del nuevo usuario:
Pepe123

Ingrese el rol del nuevo usuario (ADMIN: 1 / USUARIO: 2):
2
```

```
Run Main x
===== MENÚ CATEGORÍAS =====
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú
Seleccione:
1
```

```
Run Main x
===== MENU USUARIOS =====
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú
Seleccione:
```

```
Run Main x
0. Volver al menú
Seleccione:
1
Usuario{nombre='Pepe', apellido='Grillo', mail='pepe@gmail.com', celular='123456789', rol=USUARIO}
=== SISTEMA DE PEDIDOS (FOOD STORE) ===

---- Seleccione ----
1. Categorías
2. Productos
```

```
Run Main x
Ingrese el id de la nueva categoría:
1
Ingrese el nombre de la nueva categoría:
Snacks
Ingrese la descripción de la nueva categoría:
Comida procesada de consumo rápido
Categoría creada correctamente.
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
```

```
Run Main x
Ingrese el id del nuevo producto:
1
Ingrese el nombre:
Papas fritas
Ingrese el precio:
2500
Ingrese la descripción:
Papas fritas corte americano
```

```
Run Main x
4. Eliminar
5. Listar por categoría
0. Volver al menú
Seleccione:
1
Producto{id=1, nombre='Papas fritas', precio=2500.0, stock=25, disponible=true, categoria=Snacks}
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
```

```
Run Main x
===== MENÚ PEDIDOS =====
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menú
Seleccione:
```



```
Run Main x
Ingreso el id del nuevo pedido:
2
Ingreso el estado del pedido (1. PENDIENTE
2. CONFIRMADO
3. TERMINADO
4. CANCELADO):
1
```

```
Run Main x
3. EFECTIVO):
2
¿Desea ingresar un nuevo detalle? (Ingrese 0. Para NO, o 1. Para SI):
1
Ingreso el id del producto:
1
Ingreso la cantidad que desee del producto (Recuerde que no puede superar su stock: 25):
10
```

```
Run Main x
¿Desea ingresar un nuevo detalle? (Ingrese 0. Para NO, o 1. Para SI):
1
Ingreso el id del producto:
1
Ingreso la cantidad que desee del producto (Recuerde que no puede superar su stock: 25):
5
¿Desea ingresar un nuevo detalle? (Ingrese 0. Para NO, o 1. Para SI):
0
```

```
Run Main x
Seleccione:
1
Pedido{fecha=2026-06-21, estado=PENDIENTE, total=37500.0, formaPago=TRANSFERENCIA, usuario=Usuario{nombre='Pepe', ape
=== SISTEMA DE PEDIDOS (FOOD STORE) ===

---- Seleccione ----
1. Categorías
2. Productos
3. Usuarios
```

```
Run Main x
3. Usuarios
4. Pedidos
0. Salir

Ingrese el número correspondiente a la acción que desee ejecutar:
0

Gracias por usar el sistema. ¡Vuelva pronto!
```